

CS 320

Elijah Bastien

4/17/26

7-2 Project Two Submission

Summary

For the most part my code was aligned to the software requirements. I verified that the contact class meets some requirements and verified the contact service class meets all requirements. My main issue was with that the ContactTest class did not unit tests to verify all the validations in the Contact class. Also The TaskTest class needed unit tests to verify all the validations in the Task class but this was not fully present in my code. For the Appointment Service I completed all requirements effectively. Below is an example to highlight the ContactTest class and show why it needed improvement, this requirement was also missing in the TaskTest.

```
class ContactTest {
```

```
    @Test
```

```
    void testContactCreationSuccess() {
```

```
        Contact contact = new Contact("12345", "John", "Doe", "1234567890", "123 Main St");
```

```
        assertEquals("12345", contact.getId());
```

```
        assertEquals("John", contact.getFirstName());
```

```
        assertEquals("1234567890", contact.getPhone());
```

```
    }
```

```
    @Test
```

```
    void testContactIdTooLong() {
```

```
        assertThrows(IllegalArgumentException.class, () -> {
```

```
            new Contact("12345678901", "John", "Doe", "1234567890", "123 Main St");
```

```
        });
```

```
    }
```

```
    @Test
```

```
    void testContactPhoneInvalidLength() {
```

```
        // Test too short
```

```

assertThrows(IllegalArgumentException.class, () -> {
    new Contact("12345", "John", "Doe", "12345", "123 Main St");
});
// Test too long
assertThrows(IllegalArgumentException.class, () -> {
    new Contact("12345", "John", "Doe", "123456789012", "123 Main St");
});
}

```

@Test

```

void testContactFirstNameNull() {
    assertThrows(IllegalArgumentException.class, () -> {
        new Contact("12345", null, "Doe", "1234567890", "123 Main St");
    });
}
}

```

To defend the overall quality of my JUnit tests I looked beyond the percentage and made sure I was testing different scenarios like what happens when things go wrong or when a logic path branches off.

To you ensure that my code was technically sound I read through the instructions, wrote pseudocode, and then tried to implement that code and transform it into real code. For example, The contact service shall be able to update contact fields per contact ID. The following fields are updatable:

- firstName
- lastName
- Number
- Address
- **private** String firstName;
- **private** String lastName;
- **private** String phone;
- **private** String address;

The first step is implementing the strings and making sure they are correct. After that, I would continue.

To my code was efficient I tried my best to stick to the requirements and find the best way to execute those requirements. An example of this was using a map key in the contact service public class.

```

public class ContactService {
    // In memory storage uses a Map (Key = ID, Value = Contact Object)
    private final Map<String, Contact> contactList = new HashMap<>();

    // Add Contact with a Unique ID
    public void addContact(Contact contact) {
        if (contact == null || contactList.containsKey(contact.getId())) {
            throw new IllegalArgumentException("Contact already exists or is invalid");
        }
        contactList.put(contact.getId(), contact);
    }

    // Delete Contact by ID
    public void deleteContact(String contactId) {
        if (!contactList.containsKey(contactId)) {
            throw new IllegalArgumentException("Contact ID not found");
        }
        contactList.remove(contactId);
    }
}

```

Reflection

For the ContactService, TaskService, and AppointmentService milestones I used white box testing at the unit level. I used the Junit5 to test specific methods like testAddContact, testTaskClassSuccess, testAddAppointmentSuccess, etc. Also Boundary Value Analysis where specific tests were designed to check the exact character limits of fields such as ensuring a Task ID is no longer than 10 characters or a description does not exceed 50 characters, where also used. Another technique, Negative Testing ensured system stability and verified that the software correctly handles invalid inputs (like null values or duplicate IDs) by throwing the expected IllegalArgumentException. In the Appointment milestone, Temporal Validation was used to ensure that appointment dates are not set in the past, confirming that the time sensitive business logic is functioning correctly.

While there were many categories used in these milestones, there were also many that were not. Integration Testing was one technique not used. There is no verification of how these services interact, for example confirming an Appointment can be successfully linked to a specific Contact. Load and Performance Testing were also not used, because the services rely on HashMap structures that have not been tested for thread safety or high volume data handling. Automated Regression Testing through a CI/CD pipeline was another technique not used. This is an important technique for ensuring that new updates to one service (like Task) do not inadvertently break existing functionality in another (like

Contact). Finally, Security Testing was not performed to check for vulnerabilities beyond basic null and length validations.

These techniques have huge practical uses and implications for different software development projects and situations. Unit Testing and BVA provide a safety net that allows developers to refactor code or change internal data structures without fearing that they will break core business rules in professional environments. Negative Testing is essential for maintaining data integrity, because it rejects invalid data at the service level which avoids costly system wide crashes or primary key conflicts. The only problem is the lack of Integration Testing poses a risk for more complex projects. If these modules are combined into a single user workflow the system would likely fail.

While working on this project my mindset was trying to be as organized as possible and fulfill the requirements to the best of my ability. I tried to employ caution when I was coding my constructor of my appointment classes, checking for null values, excessive string length etc. Also while testing I covered happy path scenarios and edge cases.

```
public class AppointmentTest {  
    // Create a future date for testing (1 hour from now)  
    Date futureDate = new Date(System.currentTimeMillis() + 3600000);  
  
    @Test  
    void testAppointmentCreationSuccess() {  
        Appointment appt = new Appointment("1234567890", futureDate, "Valid description.");  
        assertEquals("1234567890", appt.getAppointmentId());  
        assertEquals(futureDate, appt.getAppointmentDate());  
        assertEquals("Valid description.", appt.getDescription());  
    }  
  
    @Test  
    void testAppointmentIdTooLong() {  
        assertThrows(IllegalArgumentException.class, () -> {  
            new Appointment("12345678901", futureDate, "Desc");  
        });  
    }  
  
    @Test  
    void testAppointmentIdNull() {
```

```

    assertThrows(IllegalArgumentException.class, () -> {
        new Appointment(null, futureDate, "Desc");
    });
}

```

@Test

```

void testAppointmentDateInPast() {
    Date pastDate = new Date(System.currentTimeMillis() - 3600000);
    assertThrows(IllegalArgumentException.class, () -> {
        new Appointment("123", pastDate, "Desc");
    });
}

```

@Test

```

void testDescriptionTooLong() {
    String longDesc = "This description is definitely longer than the allowed fifty characters
limit.";
    assertThrows(IllegalArgumentException.class, () -> {
        new Appointment("123", futureDate, longDesc);
    });
}
}

```

```

public class AppointmentServiceTest {

```

```

    Date futureDate = new Date(System.currentTimeMillis() + 3600000);

```

@Test

```

void testAddAppointmentSuccess() {
    AppointmentService service = new AppointmentService();
    Appointment appt = new Appointment("A1", futureDate, "Description");
    service.addAppointment(appt);
    assertNotNull(service.getAppointment("A1"));
}
}

```

```
}
```

```
@Test
```

```
void testAddDuplicateIdFails() {
```

```
    AppointmentService service = new AppointmentService();
```

```
    Appointment appt1 = new Appointment("A1", futureDate, "Description");
```

```
    Appointment appt2 = new Appointment("A1", futureDate, "Different Description");
```

```
    service.addAppointment(appt1);
```

```
    assertThrows(IllegalArgumentException.class, () -> {
```

```
        service.addAppointment(appt2);
```

```
    });
```

```
}
```

```
@Test
```

```
void testDeleteAppointmentSuccess() {
```

```
    AppointmentService service = new AppointmentService();
```

```
    Appointment appt = new Appointment("A1", futureDate, "Description");
```

```
    service.addAppointment(appt);
```

```
    service.deleteAppointment("A1");
```

```
    assertNull(service.getAppointment("A1"));
```

```
}
```

```
@Test
```

```
void testDeleteNonExistentIdFails() {
```

```
    AppointmentService service = new AppointmentService();
```

```
    assertThrows(IllegalArgumentException.class, () -> {
```

```
        service.deleteAppointment("NonExistent");
```

```
    });
```

```
}
```

}

While writing the code it was important to understand that the code was a web of dependencies. That is why I used state dependent testing. In terms of bias I did not really consider it an issue for this particular assignment however I did do some negative testing like testing description lengths. To finish off being disciplined in my commitment to quality as a software engineering professional was essential to me achieving 87.2% code coverage. Testing is a strenuous process, however following all steps is required to have an effective test. To avoid technical debt I think the best solution for me is to prioritize knowledge sharing and to always stay consistent in documentation.